

Sécurisation des échanges d'informations entre deux ordinateurs

Maxime LE BLAN

28798

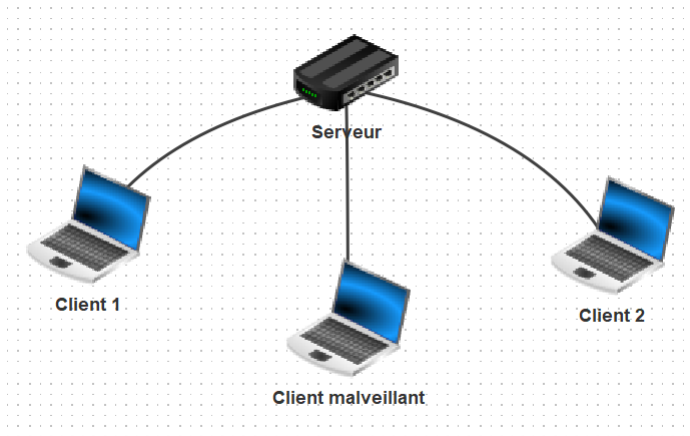
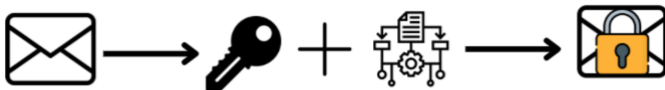
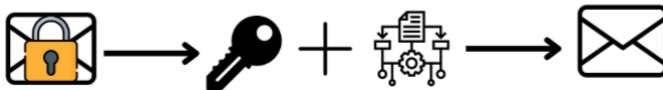


Figure: Representation imagée du système étudié

Chiffrement :



Déchiffrement :



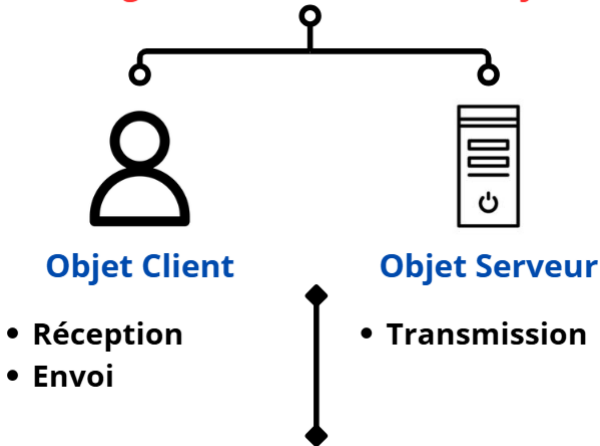
Chiffrement symétrique :



Chiffrement asymétrique :



Programmation Orientée Objet



① Problèmes :

- ▶ Programmation séquentielle ne permet pas d'envoyer et de recevoir des messages en même temps
- ▶ Envoie de données multiples ne peut se faire qu'avec des chaînes de caractères

② Solutions :

- ▶ Utilisation de fils d'exécutions
- ▶ Fonctions de conversion de liste de données chiffrées en chaînes de caractères

- ① Algorithme de chiffrement asymétrique
- ② Se base sur deux conjectures :
 - ▶ "casser" RSA par force brute nécessite de trouver la décomposition en facteurs premiers d'un nombre entier de grande taille
 - ▶ les algorithmes existants ont une complexité exponentielle en la taille de la clé

- Choix de 2 nombres premiers p et q

```
"C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms pers
Message à chiffrer : 32
Choix de p : 99999373
Choix de q : 99999437
```

- On pose $n = p \times q$ et $\phi_n = (p - 1) \times (q - 1)$

```
Valeur de n = p * q : 9999881000353001
Valeur de phi_n = (p-1) * (q-1) : 9999880800354192
```

- On prend e un nombre premier avec ϕ_n et on calcule d l'inverse modulaire de e modulo ϕ_n

```
Pour e tel que PGCD(e, phi_n) = 1, on a e : 5
Valeur de d inverse modulaire de e mod phi_n : 3999952320141677
```

- On chiffre m en calculant $m^e \pmod{n}$ avec (n, e) la clé publique et on déchiffre m' en calculant $m'^d \pmod{n}$ avec d la clé privée

```
Clé publique (n, e) : (9999881000353001, 5)
Clé privée d : 3999952320141677
Chiffrement de m : 33554432
Déchiffrement de m' : 32
```

Figure: Exécution de l'algorithme RSA sur un exemple

Protocole du modèle n°1

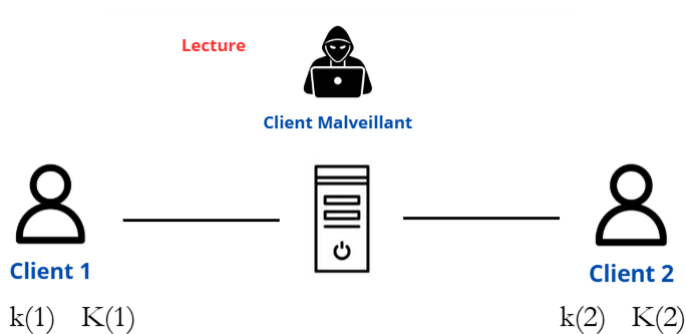


Figure: Les clients échangent leur clé publique K respective

Protocole du modèle n°1

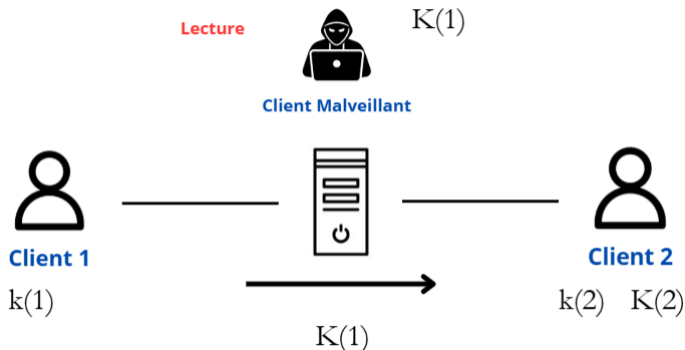


Figure: Le client 1 envoie sa clé publique $K(1)$ au client 2 en passant par le serveur

Protocole du modèle n°1

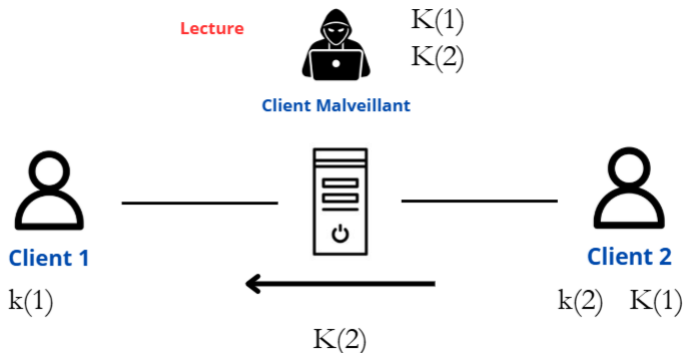


Figure: Le client 2 envoie en retour sa clé publique $K(2)$ au client 1 en passant par le serveur

Protocole du modèle n°1

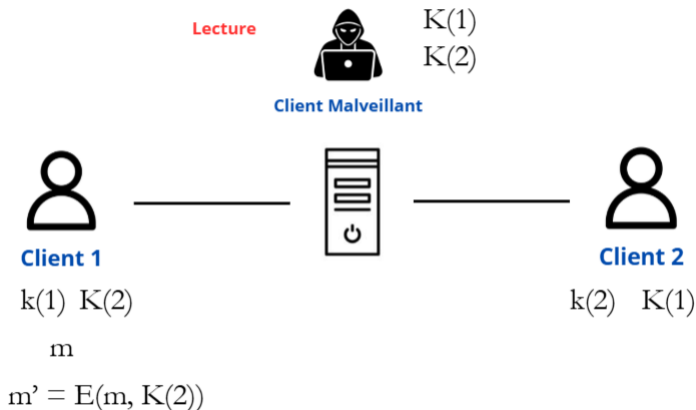


Figure: Le client 1 chiffre le message m avec la clé $K(2)$ et la fonction de chiffrement E

Protocole du modèle n°1

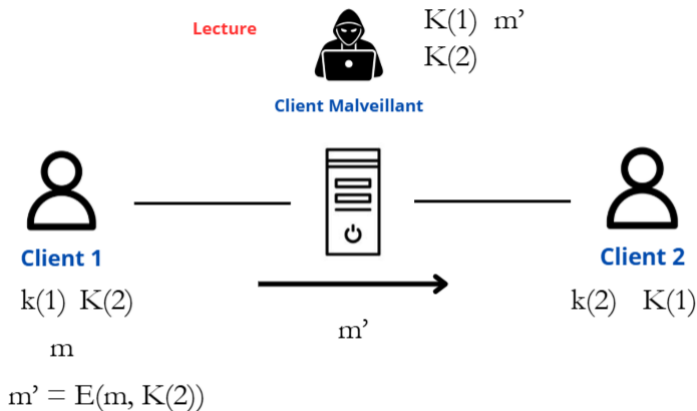


Figure: Le client 1 envoie le message chiffré m' au client 2 en passant par le serveur

Protocole du modèle n°1

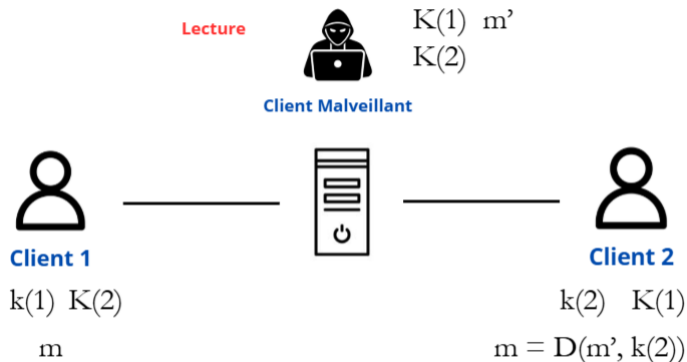
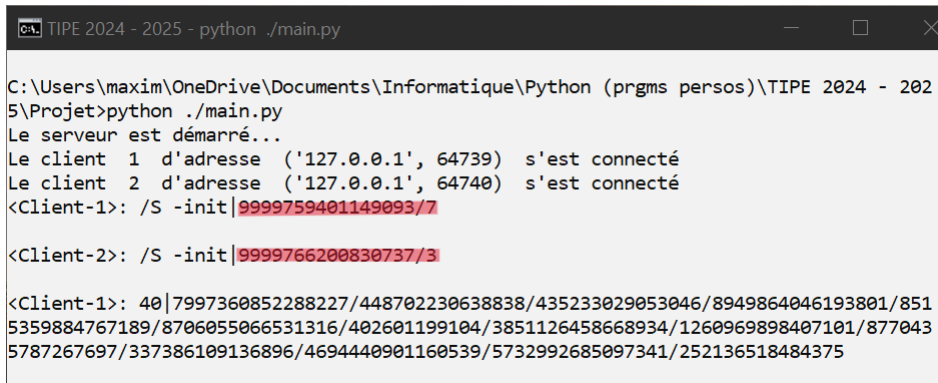


Figure: Le client 2 reçoit m' et le déchiffre avec sa clé privée $k(2)$ et la fonction de déchiffrement D

Test pratique



```
C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms persos)\TIPE 2024 - 2025\Projet>python ./main.py
Le serveur est démarré...
Le client 1 d'adresse ('127.0.0.1', 64739) s'est connecté
Le client 2 d'adresse ('127.0.0.1', 64740) s'est connecté
<Client-1>: /S -init|9999759401149093/7
<Client-2>: /S -init|9999766200830737/3
<Client-1>: 40|7997360852288227/448702230638838/435233029053046/8949864046193801/8515359884767189/8706055066531316/402601199104/3851126458668934/1260969898407101/8770435787267697/337386109136896/4694440901160539/5732992685097341/252136518484375
```

Figure: Visuel du transit du message chiffré 'Message secret écrit par Maxime LE BLAN' par le serveur, et de ce que peut voir le client malveillant

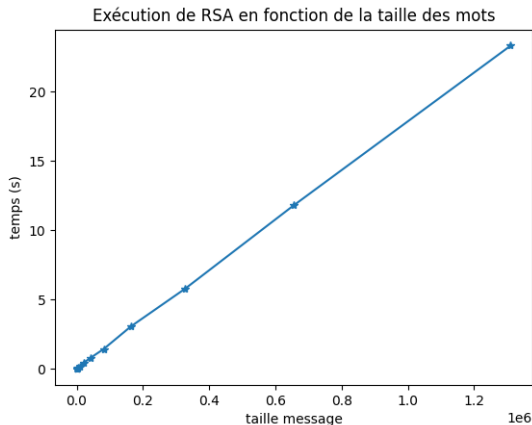
Test pratique

Taille message	temps (s)
10	0,00025
20	0,00034
40	0,00066
80	0,00133
160	0,00318
320	0,00591
640	0,01081
1280	0,02196
2560	0,04498
5120	0,0824
10240	0,17847
20480	0,3945
40960	0,76235
81920	1,40193
163840	2,83756
327680	5,64771
655360	11,34546
1310720	22,83059

Figure: Temps écoulé entre l'envoi et la réception d'un message en fonction de la taille de celui-ci avec RSA

Efficacité de l'algorithme RSA

Remarque : La complexité temporelle de l'algorithme RSA est en $O(m)$ avec m l'équivalent entier du message littéral



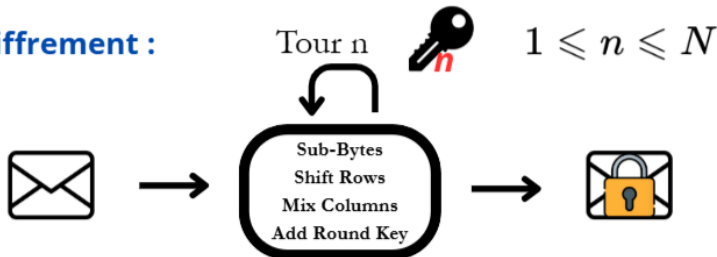
Algorithme AES



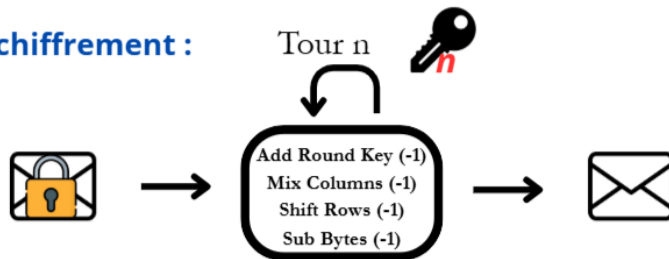
128 / 192 / 256 bits

$N \in \{10; 12; 14\}$

Chiffrement :



Déchiffrement :



Exemple avec Sub-Bytes et Shift Rows

ShiftRows :

"C:\Users\maxim\OneDrive\D

9B	01	8D	F6
74	B4	AA	4B
EF	6E	5A	F1
2C	45	92	EF

9B	01	8D	F6
4B	74	B4	AA
5A	F1	EF	6E
45	92	EF	2C

Sub Bytes:

"C:\Users\maxim\OneDrive\Documents\
paquet : 9B

paquet <- GF(paquet) : 88

paquet <- int(paquet) : 136

paquet <- bin(paquet) : 10001000

paquet <- f(paquet) : 10110001

paquet <- hex(paquet) : b1

"C:\Users\maxim\OneDrive\
9B 01 8D F6

74 B4 AA 4B

EF 6E 5A F1

2C 45 92 EF

b1 37 25 d4

d9 28 3a cb

d 9 1b ea

e7 16 4 d

Protocole du modèle n°2

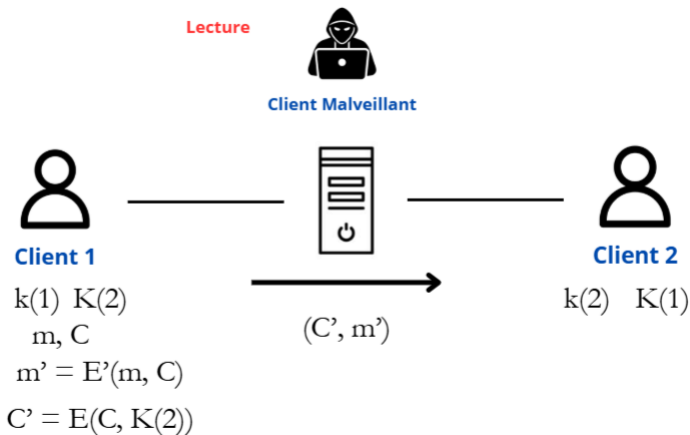
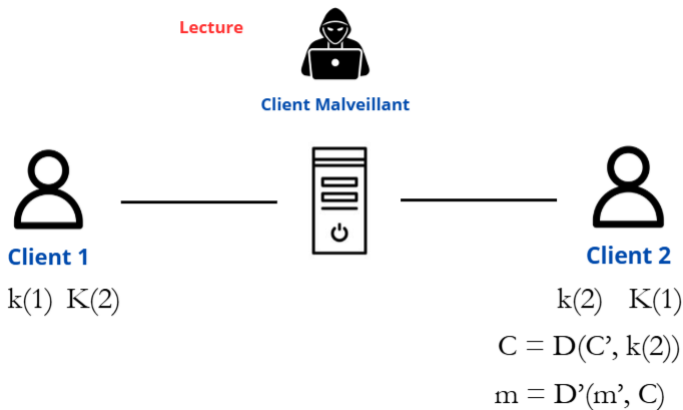


Figure: Cette fois-ci, m est chiffré avec la clé C de AES puis C est chiffrée avec $K(2)$

Protocole du modèle n°2



Test pratique

```
C:\TIPE 2024 - 2025 - python ./main.py
C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms persos)\TIPE 2024 - 2025\Projet>python ./main.py
Le serveur est démarré...
Le client 1 d'adresse ('127.0.0.1', 49236) s'est connecté
Le client 2 d'adresse ('127.0.0.1', 49237) s'est connecté
<Client-1>: /S -init|9999750801299507/13
<Client-2>: /S -init|9999805800077941/3
<Client-1>: 40|7048821731895309/3089869186329752/4077490214985222/888169166258371/75
39674587925738/4946143762801877/2150253760221988/920822653021707|433433736231969/638
1200978974952/5467127572730648/3291889827665610|s^a^i^e^ -^i^Y^V^â^?^Dox^?Ã^Ë^u^à^"}µ^J^>^
gv:öiö^ñ^tA^
```

Figure: Visuel du transit du message chiffré 'Message secret écrit par Maxime LE BLAN' par le serveur, et de ce que peut voir le client malveillant

AES + RSA

Taille message	temps (s)
10	0,00058
20	0,00061
40	0,00059
80	0,00059
160	0,00059
320	0,00059
640	0,0006
1280	0,0006
2560	0,0006
5120	0,00063
10240	0,00064
20480	0,00069
40960	0,00076
81920	0,00093
163840	0,00128
327680	0,00204
655360	0,00439
1310720	0,00783

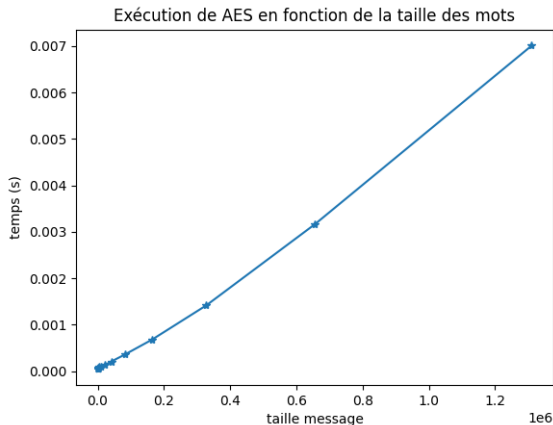
RSA

Taille message	temps (s)
10	0,00025
20	0,00034
40	0,00066
80	0,00133
160	0,00318
320	0,00591
640	0,01081
1280	0,02196
2560	0,04498
5120	0,0824
10240	0,17847
20480	0,3945
40960	0,76235
81920	1,40193
163840	2,83756
327680	5,64771
655360	11,34546
1310720	22,83059

Figure: Temps écoulé entre l'envoi et la réception d'un message en fonction de la taille de celui-ci

Efficacité de l'algorithme AES

Remarque : La complexité temporelle de l'algorithme AES est en $O(m)$ avec m l'équivalent entier du message littéral, mais AES reste en pratique plus efficace que RSA



Problème de l'Homme du milieu

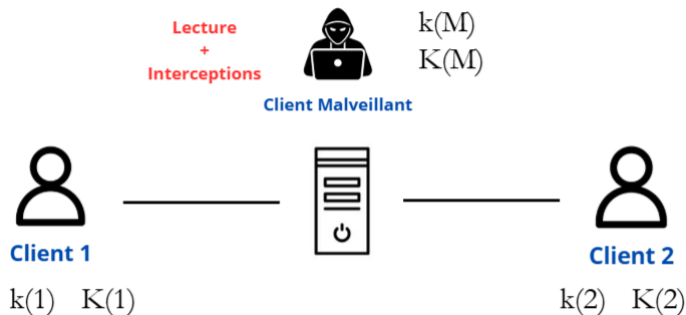


Figure: Le client malveillant crée ses propres clés $k(M)$ et $K(M)$

Problème de l'Homme du milieu

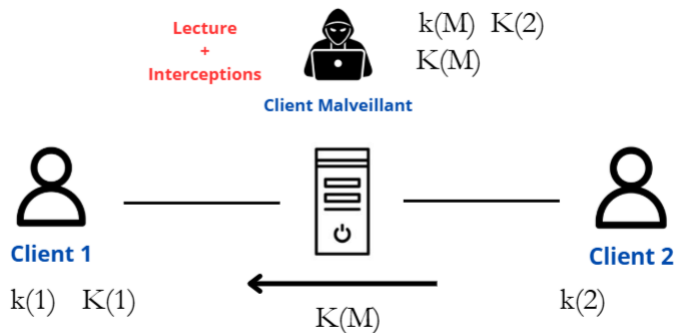


Figure: Puis il échange $K(2)$ avec $K(M)$ lors de l'envoi de $K(2)$

Problème de l'Homme du milieu

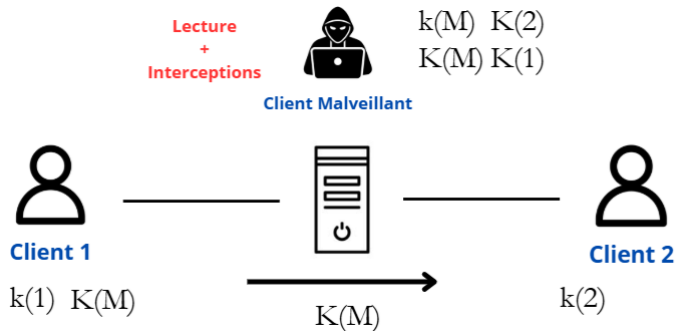


Figure: Et il échange $K(1)$ avec $K(M)$ lors de l'envoi de $K(1)$

Problème de l'Homme du milieu

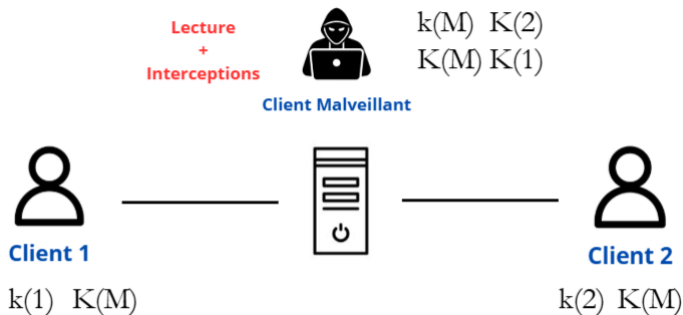
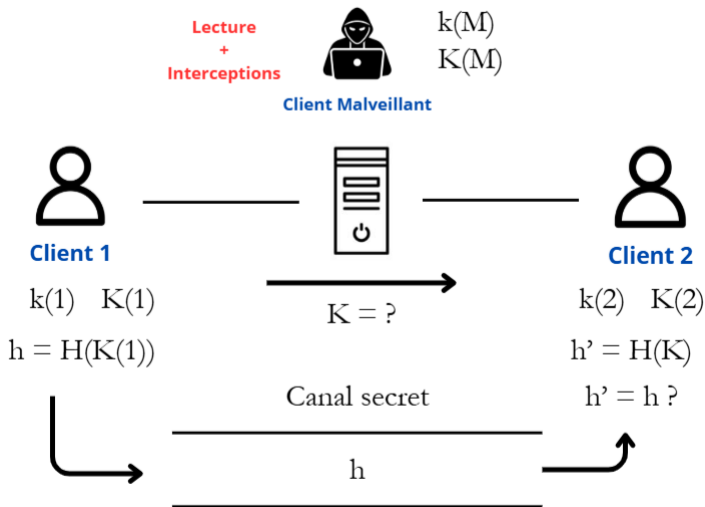


Figure: Le client malveillant peut donc lire tous les messages échangés

Protocole SAS (Short Authenticated Strings)



Test pratique

```
TIPE 2024 - 2025 - python ./intrusion.py
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms persos)\TIPE 2024 - 2025\Projet>python ./intrusion.py
clé reçue : [9999808400668763, 11]
clé envoyée : 9999768600854233/3
clé reçue : [9999770800873747, 7]
clé envoyée : 9999768600854233/3

<Client-1>:Message secret ecrit par Maxime LE BLAN
```

Figure: Interface du client malveillant après interception des clés

```
TIPE 2024 - 2025 - python ./main_c2.py

C:\Users\maxim\OneDrive\Documents\Informatique\Python (prgms persos)\TIPE 2024 - 2025\Projet>python ./main_c2.py
-->
-->
<Client-1>:Message secret ecrit par Maxime LE BLAN

-->
Comparaison des clés en cours...
La communication n'est plus sécurisée !\
```

Figure: Utilisation du canal secret par le client 1

Conclusion

- Implémentation parallélisée d'un réseau
- Implémentation d'un modèle efficace résistant à la lecture intrusive des données

Problèmes :

- Implémentation du canal secret peu sécurisée et très simplifiée
- L'intrusion du client malveillant dans la communication bloque tout échange sécurisé de données

Pistes de réflexion :

- Chercher une alternative au protocole SAS : utilisation de certificats pour l'authentification

- Soient $c, m \in \mathbb{N}$, $p, q \in \mathbb{P}$ ($p \neq q$) et $e, d \in \mathbb{N}$ tel que $ed \equiv 1 \pmod{(p-1)(q-1)}$. Posons $n = p \times q$
- Comme $c \equiv m^e \pmod{n}$, alors $c^d \equiv m^{ed} \pmod{n}$
- De plus, $ed \equiv 1 \pmod{(p-1)(q-1)}$
 $\Rightarrow \exists k \in \mathbb{N}, ed = 1 + k(p-1)(q-1)$
Donc $c^d \equiv m^{1+k(p-1)(q-1)} \pmod{n}$

- Par conséquent :
 - ▶ Si m n'est pas un multiple de p , d'après le Petit théorème de Fermat,
 $m^{p-1} \equiv 1 \pmod{p}$
 $\Rightarrow m^{ed} \equiv m^{1+k(p-1)(q-1)} \equiv m \times (m^{p-1})^{k(q-1)} \pmod{p}$
 $\Rightarrow m^{ed} \equiv m \pmod{p}$
 - ▶ Si m est un multiple de p , alors $m^{ed} \equiv 0 \pmod{p}$
 $\Rightarrow m^{ed} \equiv m \pmod{p}$

Donc $m^{ed} \equiv m \pmod{p}$ et de même, $m^{ed} \equiv m \pmod{q}$

- p et q étant premiers, on a donc :

$$m^{ed} - m \equiv 0 \pmod{pq}$$

$$\Rightarrow m^{ed} \equiv m \pmod{n}$$

$$\Rightarrow c^d \equiv m \pmod{n}$$

Propriétés :

- déterministe
- fonction de hachage avec une complexité amortie en $O(1)$
- résistance à la préimage : impossibilité, pour une valeur de hachage donnée, de construire un message ayant cette valeur
- résistance à la seconde préimage : impossibilité de trouver un second message ayant la même valeur de hachage qu'un message donné
- résistance aux collisions
- modifier légèrement un message modifie considérablement la valeur de hachage

Annexe - Modules utilisés

```
1  # Importation de modules auxiliaires de Python
2  import random as rd
3  import time
4  import threading
5  import socket
6  import sys
7  # Module utilisé pour l'utilisation d'une fonction de hachage
8  import hashlib
9  # Modules nécessaires pour l'implémentation d'AES avec des bibliothèques
10 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
11 from cryptography.hazmat.backends import default_backend
12 from cryptography.hazmat.primitives import padding
13 # Importation d'autres fichiers Python
14 import fonctions_utiles as fu
15 import algos_chiffrement as algos
```

Annexe - algo_chiffrement.py

```
1 def aux_modulo(x, n, m, acc_reste):
2     """Calcul de  $x^n \text{ modulo } m$  avec la méthode 'exponentiation rapide'"""
3     if n == 1:
4         return acc_reste % m
5     elif n % 2 == 0:
6         return aux_modulo(x*x % m, int(n/2), m, acc_reste*x % m)
7     else:
8         return aux_modulo(x*x % m, int((n-1)/2), m, acc_reste*x*x % m)
9
10 def modulo_exp(x:int, n:int, m:int):
11     """Fonction qui calcule  $x^n \text{ modulo } m$ """
12     if n == 0:
13         return 1 % m
14     else:
15         return aux_modulo(x, n, m, x)
16
17 def string_to_int(chaine: str, base=27):
18     """Fonction qui convertit une chaîne de caractères en entier et renvoie cet
19         entier"""
20     n = len(chaine)
21     entier = 0
22
23     for k in range(1, n+1):
24         entier += dico_char_a_int[chaine[k-1]] * base**(n - k)
```

Annexe - algo_chiffrement.py

```
1  def int_to_string(entier: int, taille_mot: int, b=27):
2      """Fonction qui convertit un entier en chaîne de caractères et renvoie cette chaîne"""
3
4      quotient = 0
5      diviseur = 0
6      dividende = entier
7      mot = ""
8
9      for i in range(taille_mot - 1, 0, -1):
10         diviseur = b**i
11         quotient = dividende // diviseur
12         mot += liste_int_a_char[quotient]
13         dividende -= quotient * diviseur
14         mot += liste_int_a_char[dividende%b]
15
16     return mot
17
18 # Création de l'exception "InvNotFound" utile dans la suite
19 class InvNotFound(Exception):
20     pass
```

Annexe - algo_chiffrement.py

```
1 def inverse_mod(b: int, n: int):
2     """Fonction qui prend en paramètre deux entiers b et n et renvoie l'inverse de
3         b modulo n (ie b-1)
4         ou bien renvoie l'erreur 'InvNotFound' si b-1 n'existe pas"""
5     n_0 = n
6     b_0 = b
7     t_0 = 0
8     t = 1
9     q = int(n_0/b_0)
10    r = n_0 - q * b_0
11    while r > 0:
12        temp = t_0 - q * t
13        if temp >= 0:
14            temp = temp % n
15        else:
16            temp = n - ((-temp) % n)
17        t_0 = t
18        t = temp
19        n_0 = b_0
20        b_0 = r
21        q = int(n_0/b_0)
22        r = n_0 - q * b_0
23    if b_0 != 1:
24        raise InvNotFound()
25    else:
26        return t
```

Annexe - algo_chiffrement.py

```
1 def creation_cles(lst_nb_prem: list):
2     """Fonction prenant en paramètre une liste de nombres premiers et qui crée
3     puis renvoie un couple de la forme
4     (cle_publice, cle_privée) en suivant la méthode de création des clés de l'
5     algorithme de chiffrement RSA\n
6     """
7     e = 2
8     # On choisit aléatoirement les valeurs de p et q distinctes
9     p = lst_nb_prem.pop(rd.randint(0, len(lst_nb_prem) - 1))
10    q = rd.choice(lst_nb_prem)
11    # On choisit e et phi_n
12    n = p * q
13    phi_n = (p - 1) * (q - 1)
14
15    for i in range(2, phi_n): # on cherche le premier e qui convient en partant de
16        e = 2
17        if fu.pgcd(i, phi_n) == 1:
18            e = i
19            break
20    d = inverse_mod(e, phi_n)
21
22    return ((n, e), d) # on renvoie (clé publique, clé privée)
```


Annexe - algo_chiffrement.py

```
1 def chiffrer_RSA_naif(m: int, cle_pb: tuple):
2     """Fonction qui prend en paramètre un entier m qui correspond au message à
        chiffrer et un tuple cle_pb qui
3     correspond à la clé de chiffrement. Elle renvoie un entier correspondant au
        message chiffré"""
4
5     assert (m < cle_pb[0])
6     # print("début calcul chiffrement et cle_pb : ", cle_pb)
7     reste = 1
8     for _ in range(cle_pb[1]):
9         reste *= m
10        reste = reste % cle_pb[0]
11    return reste
12
13 def chiffrer_RSA_v2(m: int, cle_pb: tuple):
14     """Fonction qui prend en paramètre un entier m qui correspond au message à
        chiffrer et un tuple cle_pb qui
15    correspond à la clé de chiffrement. Elle renvoie un entier correspondant au
        message chiffré"""
16
17    assert (m < cle_pb[0])
18    return modulo_exp(m, cle_pb[1], cle_pb[0])
```

Annexe - algo_chiffrement.py

```
1 def dechiffrer_RSA_naif(x: int, cle_pv: int, n: int):
2     """Fonction prenant en paramètre un entier x correspondant au message à déchiffrer, cle_pv un entier
3     correspondant à la clé de déchiffrement et n issue de la clé de chiffrement du message. Elle renvoie un entier
4     correspondant au message déchiffré"""
5
6     reste = 1
7     for _ in range(cle_pv):
8         reste *= x
9         reste = reste % n
10
11     return reste
12
13 def dechiffrer_RSA_v2(x: int, cle_pv: int, n: int):
14     """Fonction prenant en paramètre un entier x correspondant au message à déchiffrer, cle_pv un entier
15     correspondant à la clé de déchiffrement et n issue de la clé de chiffrement du message. Elle renvoie un entier
16     correspondant au message déchiffré"""
17
18     return modulo_exp(x, cle_pv, n)
```

Annexe - algo_chiffrement.py

```
1 def chiffrer_chaine_RSA(chaine:str, cle_pb:tuple, taille_facteur=-1):
2     """Fonction qui prend en paramètre une chaîne de caractères 'chaine' et un
3         tuple 'cle_pb' et renvoie
4         une liste d'entiers correspondants aux facteurs de taille 'taille_facteur',
5         chiffrés avec la fonction 'chiffrer_RSA(...)',
6         dont la concaténation forme la chaîne 'chaine' avant son chiffrement """
7     if taille_facteur == (-1):
8         taille_facteur = len(chaine)
9     chaine_chiffree = []
10    chaine_fractionnee = []
11    chaine_courante = ""
12
13    for i in range(len(chaine)): # On décompose 'chaine' en une liste de facteurs
14        de taille 'taille_facteur'
15        chaine_courante += chaine[i]
16        if len(chaine_courante) % taille_facteur == 0:
17            chaine_fractionnee.append(chaine_courante)
18            chaine_courante = ""
19
20    if chaine_courante != "":
21        for i in range(len(chaine_courante), taille_facteur): # on ajoute des
22            espaces pour que le dernier facteur soit de taille taille_facteur
23            chaine_courante += " "
24            chaine_fractionnee.append(chaine_courante) # on ajoute le dernier facteur
25            de taille < 'taille_facteur' (sans compter les espaces)
26
27    for facteur in chaine_fractionnee: # On chiffre tous les facteurs avec la
28        fonction 'chiffrer_RSA(...)'
29        int_facteur_courant = string_to_int(facteur, base=95)
30        chaine_chiffree.append(chiffrer_RSA_v2(int_facteur_courant, cle_pb))
31    return chaine_chiffree
```

Annexe - algo_chiffrement.py

```
1 def dechiffrer_chaine_RSA(liste:list, cle_pv:int, n:int, taille_entiers=1):
2     """_Fonction qui prend en paramètre une liste d'entiers 'liste', un entier '
3         cle_pv' et un entier n et
4         renvoie la concaténation des chaînes de caractères déchiffrées avec '
5             dechiffrer_RSA(...)' à partir des entiers
6         contenu dans 'liste' représentant des chaînes de caractères de taille '
7             taille_entiers' """
8
9     chaine_dechiffree = ""
10
11     for i in range(len(liste)): # On déchiffre tous les termes de liste sauf le
12                                 dernier
13         chaine_courante = int_to_string(dechiffrer_RSA_v2(liste[i], cle_pv, n),
14                                         taille_entiers, b=95)
15         chaine_dechiffree += chaine_courante
16
17     return chaine_dechiffree
```

Annexe - algo_chiffrement.py

```
1  def aux_bytesub(a: str):
2      """Fonction auxiliaire de bytesub qui prend en entrée un élément codé en hexad
        écimal et renvoie le résultat
3      de la fonction ByteSub appliquée à celui-ci"""
4
5      # on construit b_prime = GF(a)
6      m_gf_i = m_gf_hex_a_dec[a[0]]
7      m_gf_j = m_gf_hex_a_dec[a[1]]
8      b_prime = MATRICE_GF[m_gf_i][m_gf_j]
9      # on construit b à partir de b_prime
10     b_prime = int(b_prime, 16)
11     b_prime = str(bin(b_prime))
12     b_prime = b_prime[2:]
13     b_prime = fu.remplir_zeros(b_prime, 8)
14     b_prime = [[int(b_prime[i])] for i in range(len(b_prime))]
15     b = fu.produit_matriciel(mat_f1, b_prime, (len(mat_f1), len(mat_f1)), (len(
        b_prime), 1))
16     b = fu.somme_matricielle(b, mat_f2, len(mat_f2), 1)
17     for i in range(len(b)):
18         b[i][0] = b[i][0] % 2
19     b_final = ""
20     for chiffre in b:
21         b_final += str(chiffre[0])
22     b_final = fu.remplir_zeros(b_final, 8)
23     b_final = fu.bin_vers_dec(b_final)
24     b_final = str(hex(b_final))
25
26     return b_final[2:]
```

Annexe - algo_chiffrement.py

```
1 def bytesub(matrice: list[list]):
2     """Fonction bytesub qui prend en entrée une matrice codée en hexadécimal et
3         renvoie le résultat
4         de la fonction aux_bytesub de AES appliquée à celle-ci"""
5     return [[aux_bytesub(matrice[i][j]) for j in range(len(matrice))] for i in
6             range(len(matrice))]
7
8 def shiftrows(matrice: list[list]):
9     """Fonction qui applique la fonction ShiftRows de AES sur la matrice"""
10    n = len(matrice)
11    matrice_finale = []
12    for d in range(n):
13        matrice_finale.append(fu.decale_liste(matrice[d], d))
14    return matrice_finale
```

Annexe - algo_chiffrement.py

```
1  # Algorithme AES implémenté avec une bibliothèque
2  def chiffrer_AES(message:str, cle:bytes, vecteur:bytes):
3      """ Fonction qui prend en paramètre un message, une clé et un vecteur et
4          renvoie une chaîne de
5          caractères correspondant au chiffrement de 'message' avec l'algorithme AES en
6          utilisant la clé 'cle'
7          et le vecteur d'initialisation 'vecteur' \n
8          message: chaîne de caractères \n
9          cle: chaîne de caractères sur 32 octets \n
10         vecteur: chaîne de caractères sur 16 octets """
11
12         padder = padding.PKCS7(128).padder()
13         padded_data = padder.update(message.encode('utf-8'))
14         padded_data += padder.finalize()
15
16         cipher = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=
17             default_backend())
18         encryptor = cipher.encryptor()
19         message_chiffre = encryptor.update(padded_data) + encryptor.finalize()
20
21         return message_chiffre
```

Annexe - algo_chiffrement.py

```
1  def dechiffrer_AES(message_chiffre, cle:bytes, vecteur:bytes):
2      """Fonction qui prend en paramètre un message, une clé et un vecteur et
          renvoie une chaîne de
3          caractères correspondant au déchiffrement de 'message_chiffre' avec l'
          algorithme AES en utilisant la clé 'cle'
4          et le vecteur d'initialisation 'vecteur'\n
5          message_chiffre: chaîne de caractères\n
6          cle: chaîne de caractères sur 32 octets\n
7          vecteur: chaîne de caractères sur 16 octets"""
8
9      decryptor = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=
          default_backend()).decryptor()
10     decrypted_data = decryptor.update(message_chiffre) + decryptor.finalize()
11
12     unpadder = padding.PKCS7(128).unpadder()
13     unpadded_data = unpadder.update(decrypted_data)
14     unpadded_data += unpadder.finalize()
15
16     return unpadded_data.decode('utf-8')
```



```
1  class Serveur:
2      """Classe permettant de simuler un serveur informatique simplifié à l'aide du
        module socket"""
3
4      def __init__(self, modele:int, port=6390):
5          self.host = ""
6          self.port = port
7          self.socket = None
8          self.clients = []
9          self.threads = {}
10         self.id_client = 1
11         self.arret = False
12         self.modele = modele
13
14     def close(self):
15         """Fonction qui vérifie si le serveur doit s'éteindre ou non"""
16         while True:
17             if self.arret == True:
18                 break
```

```
1      def connections(self):
2          """Fonction qui gère les connexions au serveur d'éventuels clients en
           temps réel"""
3          while True:
4              client, ip = self.socket.accept()
5              self.clients.append((self.id_client, client, ip))
6              # on lance un fil d'exécution pour chaque nouveau client afin de
                pouvoir recevoir ses messages
7              # on choisit la bonne fonction de réception en fonction du modèle
                utilisé
8              if self.modele != 3:
9                  self.threads[str(self.id_client)] = threading.Thread(target=self.
                    reception, args=[client, self.id_client])
10             else:
11                 self.threads[str(self.id_client)] = threading.Thread(target=self.
                    reception_cm, args=[client, self.id_client])
12                 self.threads[str(self.id_client)].start()
13
14             print("Le client", self.id_client, "d'adresse", ip, "s'est connecté
                ")
15             self.id_client += 1
16
17     def transfert(self, expéditeur:str, destinataire, message:str, separateur=":"):
18         """Fonction qui transfert un message entre 2 clients"""
19         new_msg = "<Client-" + expéditeur + ">" + separateur + message
20         destinataire.send(new_msg.encode("utf-8"))
```

```
1  def reception(self, client, id_client:int):
2      """Fonction qui réceptionne les requêtes des clients et qui les traite
        pour les modèles 1 et 2"""
3      while True:
4          requete_client = client.recv(500)
5          requete_client = requete_client.decode("utf-8")
6          print("<Client-" + str(id_client) + ">", requete_client)
7          print() # on passe une ligne pour aérer l'affichage
8          if requete_client == "/S_close":
9              print("CLOSE")
10             self.arret = True
11         else: # on envoie le message au bon destinataire en fonction de l'expé-
                diteur
12             if id_client == 1:
13                 self.transfert(str(id_client), self.clients[1][1],
                    requete_client, separateur="|")
14             else:
15                 self.transfert(str(id_client), self.clients[0][1],
                    requete_client, separateur="|")
```

```
1  def reception_cm(self, client, id_client:int):
2      """Fonction qui réceptionne les requêtes des clients et qui les traite
        pour le modèle 3"""
3      while True:
4          requete_client = client.recv(500)
5          requete_client = requete_client.decode("utf-8")
6          decomp_msg = fu.decomposition_chaine(requete_client, separateur="|")
7          print("<Client-" + str(id_client) + ">:", requete_client)
8          print() # on passe une ligne pour aérer l'affichage
9          if requete_client == "/S_close":
10             print("CLOSE")
11             self.arret = True
12         elif "/S_check" in requete_client:
13             if id_client == 1:
14                 destinataire = 1
15             else:
16                 destinataire = 0
17             self.transfert(str(id_client), self.clients[destinataire][1],
                            requete_client, separateur="|")
```

Annexe - serveur.py

```
1         elif "/S_--init" in requete_client:
2             if len(decomp_msg) > 2: # si le client malveillant veut envoyer une
3                                     # clé altérée
4                                     # on l'envoie au bon destinataire
5                 expéditeur = decomp_msg[2]
6                 destinataire = int(decomp_msg[3]) - 1
7                 # on supprime les infos concernant l'expéditeur et le
8                 # destinataire
9                 decomp_msg.pop()
10                decomp_msg.pop()
11                requete_client = fu.compression_arg_str(decomp_msg, separateur="|")
12                self.transfert(expéditeur, self.clients[destinataire][1],
13                               requete_client, separateur="|")
14            else: # sinon c'est la clé d'origine d'un client normal
15                # on envoie donc sa clé au client malveillant
16                self.transfert(str(id_client), self.clients[2][1],
17                               requete_client, separateur="|")
18            elif id_client != 3: # si le serveur reçoit un message d'un client
19                                # normal
20                # on l'envoie au client malveillant
21                self.transfert(str(id_client), self.clients[2][1], requete_client,
22                               separateur="|")
23            else:
24                # on l'envoie au bon destinataire
25                expéditeur = decomp_msg[4]
26                destinataire = int(decomp_msg[5]) - 1
27                # on supprime les infos concernant l'expéditeur et le destinataire
28                decomp_msg.pop()
29                decomp_msg.pop()
30                requete_client = fu.compression_arg_str(decomp_msg, separateur="|")
31                self.transfert(expéditeur, self.clients[destinataire][1],
```

```
1     def run(self):
2         """Fonction qui démarre et ferme le serveur"""
3         # création du socket pour "démarrer" le serveur
4         self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5         self.socket.bind((self.host, self.port))
6         print("Le serveur est démarré...")
7
8         self.socket.listen(1) # on le met en mode récepteur
9
10        connections = threading.Thread(target=self.connections, args=[])
11        connections.start()
12
13        execution = threading.Thread(target=self.close, args=[])
14        execution.start()
15
16        execution.join()
17
18        for client in self.clients:
19            client[1].close()
20        self.socket.close()
```

```
1 class Client_RSA:
2     """Classe permettant de simuler un client pouvant se connecter à un serveur
3     informatique simplifié
4     à l'aide du module socket, qui base le chiffrement sur l'algorithme du RSA"""
5     def __init__(self, id: int, liste_nb_premiers, host="localhost", port=6390):
6         self.id = id
7         self.host = host
8         self.port = port
9         self.socket = None
10        self.cle_pv_RSA = None
11        self.cle_pb_RSA = None
12        self.cles_pb_RSA_autres = {}
13        self.liste_nb_premiers = liste_nb_premiers
14        self.init_envoi = False # permet de savoir si self.initialisation_RSA() a d
15                               # déjà été exécutée dans self.envoyer()
16        self.init_recu = False # permet de savoir si self.initialisation_RSA() a d
17                               # déjà été exécutée dans self.reception()
18        # messages envoyés de la forme : "Client_X:taille_msg:message_chiffre"
19        # On renomme les indices que l'on utilise dans le code pour pouvoir les
20        # modifier plus facilement
21        # et pour plus de visibilité
22        self.i_client = 0
23        self.i_taille_msg = 1
24        self.i_msg_chiffre = 2
```

```
1  def initialisation_RSA(self, message: str, separateur="|"):
2      """Fonction qui crée la clé privée et la clé publique du client pour l'
          algo de chiffrement RSA.
3      Elle est exécutée à l'initialisation de la connection avec le serveur et
          partage la clé publique du client
4      avec un autre client connecté au serveur avec lequel il veut communiquer."""
5
6      cles = algos.creation_cles(self.liste_nb_premiers)
7      self.cle_pb_RSA = cles[0]
8      self.cle_pv_RSA = cles[1]
9      msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
10     msg = message + separateur + msg
11
12     return msg
13
14     def envoyer(self, socket):
15         """Fonction permettant d'envoyer un message à un client en passant par le
            serveur"""
16         try:
17             while True:
18                 msg = input("-->")
19                 if msg == "/S_init" and not self.init_envoi:
20                     socket.send(self.initialisation_RSA(msg).encode("utf-8"))
21                     self.init_envoi = True
22                 elif msg == "/S_close":
23                     socket.send(msg.encode("utf-8"))
```



```
1         else:
2             try:
3                 if self.id == 1: # on chiffre le message avec la clé
4                     # publique du bon client
5                     # msg_chiffre = algos.chiffrer_RSA(algos.string_to_int(
6                     #     msg), self.cles_pb_RSA_autres["2"])
7                     msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.
8                         cles_pb_RSA_autres["2"], taille_facteur=3)
9                 else:
10                    # msg_chiffre = algos.chiffrer_RSA(algos.string_to_int(
11                    #     msg), self.cles_pb_RSA_autres["1"])
12                    msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.
13                        cles_pb_RSA_autres["1"], taille_facteur=3)
14                # on transforme le message sous forme de liste en chaîne de
15                # caractères
16                msg_chiffre = fu.compression_arg_str(msg_chiffre)
17                msg_envoye = str(len(msg)) + "|" + str(msg_chiffre) # on
18                # ajoute la longueur du message chiffré au message envoy
19                # é
20                msg_envoye = msg_envoye.encode("utf-8")
21                socket.send(msg_envoye)
22            except AssertionError:
23                print("Erreur lors de la tentative de chiffrement du
24                    message. Veuillez essayer à nouveau")
25        except ConnectionResetError:
26            print("Déconnexion du serveur...")
27            sys.exit()
```

```
1  def reception(self, socket):
2      """Fonction qui réceptionne les messages reçus du serveur"""
3      try:
4          while True:
5              requete_serveur = socket.recv(500)
6              requete_serveur = requete_serveur.decode("utf-8")
7              # requete_serveur est de la forme : "Client_X:taille_msg:
               message_chiffre"
8              decomp_msg = fu.decomposition_chaine(requete_serveur)
9              id_expediteur = fu.id_client(requete_serveur)
10
11             if "/S_-init" in decomp_msg and not self.init_recu: # si on a pas
               enregistré les clés publiques de l'autre client, on le fait
12                 self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(
                   decomp_msg[self.i_msg_chiffre])
13                 self.init_recu = True
14                 if not self.init_envoi: # si on a pas échangé notre clé
                   publique, on le fait
15                     socket.send(self.initialisation_RSA("/S_-init").encode("utf
                       -8"))
16                 self.init_envoi = True
```

```
1         else:
2             taille_msg = int(decomp_msg[self.i_taille_msg])
3             # on reconvertis le message chiffré (str) en liste d'entiers
4             pour pouvoir ensuite le déchiffrer
5             msg_chiffre = fu.extraction_arg_str(decomp_msg[self.
6                 i_msg_chiffre])
7             # on vérifie qui est l'expéditeur pour prendre sa clé publique
8             correspondante
9             if id_expediteur == 1:
10                 # msg_dechiffre = algos.dechiffrer_RSA(int(decomp_msg[2]),
11                     self.cle_pv_RSA, self.cle_pb_RSA[0])
12                 msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre,
13                     self.cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=3)
14             else:
15                 msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre,
16                     self.cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=3)
17             print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)
18             print("\n-->", end="")
19 except ConnectionResetError:
20     print("Connexion interrompue\nSi le terminal est en attente, appuyez\nsur Entrée")
21     sys.exit()
```

Annexe - client.py

```
1  def connection(self):
2      """Fonction permettant de connecter le client à un serveur"""
3      self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4      self.socket.connect((self.host, self.port))
5      envoi = threading.Thread(target=self.envoyer, args=[self.socket])
6      recep = threading.Thread(target=self.reception, args=[self.socket])
7      envoi.start()
8      recep.start()
9
10 class Client_AES_RSA(Client_RSA):
11     """Classe fille de Client_RSA permettant de simuler un client pouvant se
12         connecter à un serveur informatique simplifié
13         à l'aide du module socket, qui base le chiffrement sur une combinaison de l'
14         algorithme RSA et AES"""
15     def __init__(self, id: int, liste_nb_premiers):
16         Client_RSA.__init__(self, id, liste_nb_premiers)
17         # messages envoyés de la forme : "Client_X/taille_msg/cle_AES/vect_AES/
18         # message_chiffre"
19         # On renomme les indices que l'on utilise dans le code pour pouvoir les
20         # modifier plus facilement
21         # et pour plus de visibilité
22         self.i_client = 0
23         self.i_taille_msg = 1
24         self.i_cle_AES = 2
25         self.i_vect_AES = 3
26         self.i_msg_chiffre = 4
```

```
1  def envoyer(self, socket):
2      """Fonction permettant d'envoyer un message à un client en passant par le
        serveur"""
3      try:
4          while True:
5              msg = input("-->")
6              if msg == "/S-init" and not self.init_envoi:
7                  # on envoie notre clé publique à l'autre client
8                  socket.send(self.initialisation_RSA(msg, separateur="|").encode(
                    "utf-8"))
9                  self.init_envoi = True
10             elif msg == "/S-close":
11                 socket.send(msg.encode("utf-8"))
12             elif msg == "/S-check":
13                 # on construit le hash de la clé publique de l'expéditeur et on
                    l'envoie à l'autre client
14                 cle_pb = str(self.cle_pb_RSA[0]) + "/" + str(self.cle_pb_RSA
                    [1])
15                 hash_cle = hashlib.sha256(cle_pb.encode())
16                 hash_cle = hash_cle.hexdigest()
17                 msg += "|" + hash_cle
18                 socket.send(msg.encode())
19             else:
20                 try:
21                     cle_AES = fu.generation_alea_chaine(32)
22                     vecteur_AES = fu.generation_alea_chaine(16)
23                     # si on veut envoyer un msg au client 1, on chiffre avec la
                        clé publique du client 2
24                     if self.id == 1: # on chiffre le message avec la clé
                        publique du bon client
25                         destinataire = "2"
```

```
1         else: # sinon c'est l'inverse
2             destinataire = "1"
3             # On chiffre le message avec la cle et le vecteur AES
4             msg_chiffre = algos.chiffrer_AES(msg, fu.encode_ascii(
5                 cle_AES), fu.encode_ascii(vecteur_AES))
6             msg_chiffre = msg_chiffre.decode("latin-1")
7             # On chiffre la cle et le vecteur pour AES avec le RSA
8             cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
9                 cles_pb_RSA_autres[destinataire], taille_facteur=4)
10            vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES, self.
11                cles_pb_RSA_autres[destinataire], taille_facteur=4)
12            # on transforme le message, la clé et le vecteur sous forme
13            # de liste en chaine de caractères
14            msg_chiffre = fu.compression_arg_str([msg_chiffre])
15            cle_AES = fu.compression_arg_str(cle_AES)
16            vecteur_AES = fu.compression_arg_str(vecteur_AES)
17            # On construit le message que l'on va envoyer contenant
18            # toutes les infos nécessaires pour le destinataire
19            msg_envoye = str(len(msg)) + "|" + cle_AES + "|" +
20                vecteur_AES + "|" + msg_chiffre
21            msg_envoye = msg_envoye.encode("utf-8")
22            socket.send(msg_envoye)
23
24    except AssertionError:
25        print("Erreur lors de la tentative de chiffrement du
26            message. Veuillez essayer à nouveau")
27
28    except ConnectionResetError:
29        print("Déconnexion du serveur...")
30        sys.exit()
```

```
1      def reception(self, socket):
2          """Fonction qui réceptionne les messages reçus du serveur"""
3          try:
4              while True:
5                  requete_serveur = socket.recv(500)
6                  requete_serveur = requete_serveur.decode("utf-8")
7                  decomp_msg = fu.decomposition_chaine(requete_serveur, separateur="|")
8                  id_expediteur = fu.id_client(requete_serveur)
9                  if "/S-init" in decomp_msg and not self.init_recu: # si on a pas
enregistré les clés publiques de l'autre client, on le fait
10                     self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(
decomp_msg[2])
11                     self.init_recu = True
12                     if not self.init_envoi: # si on a pas échangé notre clé
publique, on le fait
13                         socket.send(self.initialisation_RSA("/S-init", separateur=
"|").encode("utf-8"))
14                         self.init_envoi = True
15                     elif "/S-check" in requete_serveur:
16                         print("\nComparaison des clés en cours...")
17                         # on construit le hash de la clé de l'autre client à partir de
notre base de donnée
18                         cle_pb = str(self.cles_pb_RSA_autres[id_expediteur][0]) + "/" +
str(self.cles_pb_RSA_autres[id_expediteur][1])
19                         hash_cle = hashlib.sha256(cle_pb.encode())
20                         hash_cle = hash_cle.hexdigest()
21                         # on récupère le hash envoyé par l'autre client
22                         hash_autre_cle = decomp_msg[-1]
```

```
1         if hash_autre_cle != hash_cle:
2             print("La communication n'est plus sécurisée !\n")
3         else:
4             print("La communication est sécurisée")
5     else:
6         # On extrait et déchiffre la clé et le vecteur AES
7         cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
8         cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.cle_pv_RSA,
9             self.cle_pb_RSA[0], taille_entiers=4)
10        vecteur_AES = fu.extraction_arg_str(decomp_msg[self.i_vect_AES])
11        vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
12            cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
13
14        # on récupère le message chiffré et on le déchiffre
15        msg_chiffre = decomp_msg[self.i_msg_chiffre]
16        msg_chiffre = msg_chiffre.encode("latin-1")
17        msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
18            encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
19        print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)
20        print("\n-->", end="")
21    except ConnectionResetError:
22        print("Connexion interrompue\nSi le terminal est en attente, appuyez
23            sur Entrée")
24    sys.exit()
```


Annexe - client.py

```
1 class ClientMalveillant(Client_AES_RSA):
2     """Classe fille de la classe Client_AES_RSA qui initialise le client
3         malveillant voulant lire des conversations
4         sécurisées entre les autres clients connectés au même serveur, dans le cadre du
5         modèle n 3 avec la fonction
6         de hachage et le canal secret"""
7     def __init__(self, id: int, liste_nb_premiers):
8         Client_AES_RSA.__init__(self, id, liste_nb_premiers)
9         # Compteur indiquant le nombre d'échanges de clés que le Client Malveillant
10         a effectué avec les autres clients
11         self.nb_cles_echangeées = 0
12
13     def initialisation_RSA(self, message: str, separateur="|", expéditeur="",
14        destinataire=""):
15         """Fonction qui crée la clé privée et la clé publique du client pour l'
16             algo de chiffrement RSA.
17             Elle est exécutée à l'initialisation de la connection avec le serveur et
18             partage la clé publique du client
19             avec un autre client connecté au serveur avec lequel il veut communiquer."""
20         if self.cle_pv_RSA == None: # si on a pas encore créé nos clés, on le fait
21             cles = algos.creation_cles(self.liste_nb_premiers)
22             self.cle_pb_RSA = cles[0]
23             self.cle_pv_RSA = cles[1]
24             msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
25             print("clé envoyée:", msg)
26             msg = fu.compression_arg_str([msg, expéditeur, destinataire], separateur="|")
27             msg = message + separateur + msg
28             return msg
```

```
1  def reception(self, socket):
2      """Fonction qui réceptionne les messages reçus du serveur"""
3      try:
4          while True:
5              requete_serveur = socket.recv(500)
6              requete_serveur = requete_serveur.decode("utf-8")
7              decomp_msg = fu.decomposition_chaine(requete_serveur, separateur="|")
8              id_expediteur = fu.id_client(requete_serveur)
9
10             if "/S-init" in decomp_msg: # si on reçoit une clé publique, on l'enregistre
11                 self.cles_pb_RSA_autres[id_expediteur] = fu.extraction_arg_str(decomp_msg[2])
12                 print("clé reçue:", fu.extraction_arg_str(decomp_msg[2]))
13                 # on envoie notre propre clé publique au bon destinataire
14                 if id_expediteur == "1":
15                     socket.send(self.initialisation_RSA("/S-init", separateur="|", expediteur=id_expediteur, destinataire="2").encode("utf-8"))
16                 else:
17                     socket.send(self.initialisation_RSA("/S-init", separateur="|", expediteur=id_expediteur, destinataire="1").encode("utf-8"))
```

```
1         else:
2             # On extrait et déchiffre la clé et le vecteur AES
3             cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
4             cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.cle_pv_RSA,
5                 self.cle_pb_RSA[0], taille_entiers=4)
6             vecteur_AES = fu.extraction_arg_str(decomp_msg[self.i_vect_AES
7                 ])
8             vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
9                 cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
10
11             # on récupère le message chiffré et on le déchiffre
12             msg_chiffre = decomp_msg[self.i_msg_chiffre]
13             msg_chiffre = msg_chiffre.encode("latin-1")
14             msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
15                 encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
16             print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre)
17
18             # on renvoie le message au destinataire chiffré avec la clé
19             publique du destinataire
20             # si on veut envoyer un msg au client 1, on chiffre avec la clé
21             publique du client 2
22             if id_expediteur == "1": # on chiffre le message avec la clé
23                 publique du bon client
24                 destinataire = "2"
25             else: # sinon c'est l'inverse
26                 destinataire = "1"
```

```
1      # On chiffre le message avec la cle et le vecteur AES
2      msg_chiffre = algos.chiffrer_AES(msg_dechiffre, fu.encode_ascii
      (cle_AES), fu.encode_ascii(vecteur_AES))
3      msg_chiffre = msg_chiffre.decode("latin-1")
4      # On chiffre la cle et le vecteur pour AES avec le RSA
5      cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
      cles_pb_RSA_autres[destinataire], taille_facteur=4)
6      vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES, self.
      cles_pb_RSA_autres[destinataire], taille_facteur=4)
7      # on transforme le message, la clé et le vecteur sous forme de
      liste en chaine de caractères
8      msg_chiffre = fu.compression_arg_str([msg_chiffre])
9      cle_AES = fu.compression_arg_str(cle_AES)
10     vecteur_AES = fu.compression_arg_str(vecteur_AES)
11     # On construit le message que l'on va envoyer contenant toutes
      les infos nécessaires pour le destinataire
12     msg_envoye = str(len(msg_dechiffre)) + "|" + cle_AES + "|" +
      vecteur_AES + "|" + msg_chiffre + "|" + id_expediteur + "|"
      + destinataire
13     msg_envoye = msg_envoye.encode("utf-8")
14     socket.send(msg_envoye)
15 except ConnectionResetError:
16     print("Connexion interrompue\nSi le terminal est en attente, appuyez
      sur Entrée")
17     sys.exit()
18
19 def connection(self):
20     """Fonction permettant de connecter le client à un serveur"""
21     self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
22     self.socket.connect((self.host, self.port))
23     recep = threading.Thread(target=self.reception, args=[self.socket])
24     recep.start()
```

Annexe - fonctions _utiles.py

```
1  def copie_dict(dico:dict):
2      """Fonction qui renvoie une copie indépendante du dictionnaire 'dico'"""
3      dico_copie = {}
4      for cle, valeur in dico.items():
5          dico_copie[cle] = valeur
6
7      return dico_copie
8
9  def pgcd(a: int, b: int):
10     """Fonction qui prend en paramètre 2 entiers naturels a et b et renvoie leur PGCD"""
11
12     dividende = max(a, b)
13     diviseur = min(a, b)
14     temp = 0
15     while (dividende % diviseur) != 0 :
16         temp = dividende % diviseur
17         dividende = diviseur
18         diviseur = temp
19
20     return diviseur
```

Annexe - fonctions _utiles.py

```
1 def compression_arg_str(liste_args: list, separateur="/"):
2     """Fonction qui prend en paramètre une liste d'objets pouvant être convertis
        en chaîne de caractères, un séparateur '/' et renvoie une chaîne de caractères
        contenant n paramètres et qui est de la forme 'arg[1]/arg[2]/.../arg[n]' """
3     arguments = ""
4     for i in range(len(liste_args)):
5         if i != len(liste_args) - 1:
6             arguments += str(liste_args[i]) + separateur
7         else:
8             arguments += str(liste_args[i])
9     return arguments
10
11 def extraction_arg_str(chaine_args: str, separateur="/"):
12     """Fonction qui prend en paramètre une chaîne de caractères contenant n
        arguments et de la forme 'arg[1]/arg[2]/.../arg[n]', un séparateur '/' et
        renvoie une liste contenant les n arguments de type int """
13     chiffres = "0123456789"
14     liste_args = []
15     arg_courant = ""
16     for i in range(len(chaine_args)):
17         if chaine_args[i] != separateur and chaine_args[i] in chiffres:
18             arg_courant += chaine_args[i]
19         else:
20             liste_args.append(int(arg_courant))
21             arg_courant = ""
22     if arg_courant != "":
23         liste_args.append(int(arg_courant))
24     return liste_args
```

Annexe - fonctions_utiles.py

```
1 def decomposition_chaine(chaine: str, separateur="|"):
2     """Fonction qui prend en paramètre une chaîne de caractères que l'on peut
        diviser en sous-chaînes séparées par un séparateur: 'et renvoie une
        liste contenant ces sous-chaînes"""
3     liste_sous_chaines = []
4     arg_courant = ""
5     for i in range(len(chaine)):
6         if chaine[i] != separateur:
7             arg_courant += chaine[i]
8         else:
9             liste_sous_chaines.append(arg_courant)
10            arg_courant = ""
11    if arg_courant != "":
12        liste_sous_chaines.append(arg_courant)
13    return liste_sous_chaines
14
15 def id_client(chaine: str):
16     """Fonction qui prend une chaîne de caractères de type '<Client-*>***' et
        renvoie l'entier * en chaîne de caractères"""
17     chiffres = "0123456789"
18     id_passe = False
19     id = ""
20     for char in chaine:
21         if char in chiffres:
22             id_passe = True
23             id += char
24         elif id_passe:
25             break
26    return id
```

Annexe - fonctions _utiles.py

```
1  def importer_fichier(nom_fichier:str, type="int"):  
2      """ Importe un fichier texte et ajoute chaque ligne du fichier dans une liste  
        Python que la fonction renvoie """  
3      liste_nb_premiers = []  
4      with open(nom_fichier, "r") as fichier:  
5          contenu = fichier.readlines()  
6      for ligne in contenu:  
7          ligne_actuelle = ligne.rstrip("\n")  
8          if type == "int":  
9              liste_nb_premiers.append(int(ligne_actuelle))  
10             else:  
11                 liste_nb_premiers.append(ligne_actuelle)  
12      return liste_nb_premiers  
13  
14  def generateur_alea_chaine(taille_chaine:int):  
15      """ Fonction qui génère aléatoirement une chaîne de caractères de taille '  
        taille_chaine', composée des lettres de l'alphabet en minuscule, et la  
        renvoie """  
16      liste_int_a_char = "abcdefghijklmnopqrstuvwxyz"  
17      nb_char = len(liste_int_a_char)  
18      chaine_alea = ""  
19      for _ in range(taille_chaine):  
20          i = rd.randint(0, nb_char - 1)  
21          chaine_alea += liste_int_a_char[i]  
22      return chaine_alea
```


Annexe - fonctions __utiles.py

```
1  def concat_chaine(chaine:str, n:int):
2      """_Fonction_ qui_ renvoie_ la_ concaténation_ de_ 'chaine' _ (n-1) _ fois _"""
3
4      chaine_finale = ""
5      for _ in range(n):
6          chaine_finale += chaine
7
8      return chaine_finale
9
10 def encode_ascii(chaine:str):
11     """_Fonction_ qui_ transforme_ une_ chaine_ str_ en_ bstr_ (ie_ en_ code_ ASCII) \n
12     NB_ :_ elle_ fait_ la_ même_ chose_ que_ la_ fonction_ native_ bytes(chaine, _encoding="
13         ascii")_"""
14
15     return bytes(chaine, encoding="utf-8")
```

Annexe - fonctions _utiles.py

```
1  def matrice_gf(chaine: str, taille=16):
2      """Renvoi la matrice GF sous forme de matrice de type list[list]"""
3      matrice_gf = decomposition_chaine(chaine, separateur="\n")
4      matrice_gf_bis = []
5      matrice_gf_finale = []
6      ligne_gf = []
7
8      for i in range(len(matrice_gf)):
9          if "\n" in matrice_gf[i]:
10             matrice_gf_bis.append(matrice_gf[i][:(-1)])
11          elif matrice_gf[i] != "":
12             matrice_gf_bis.append(matrice_gf[i])
13
14      for i in range(taille):
15          for j in range(taille):
16             ligne_gf.append(matrice_gf_bis[i*taille + j])
17             if j == taille - 1:
18                 matrice_gf_finale.append(ligne_gf)
19                 ligne_gf = []
20
21      return matrice_gf_finale
```

Annexe - fonctions_utiles.py

```
1 def conversion_matrice(matrice: list[list], fonction, base=10):
2     """Fonction qui applique la fonction de conversion 'fonction' sur chaque case
3     de la matrice carrée"""
4     res = matrice.copy()
5     for i in range(len(matrice)):
6         for j in range(len(matrice)):
7             if base != 10:
8                 res[i][j] = fonction(res[i][j], base)
9             else:
10                 res[i][j] = fonction(res[i][j])
11     return res
```

Annexe - fonctions __utiles.py

```
1  def produit_matriciel(m1: list[list[int]], m2: list[list[int]], n1=(-1, -1), n2
   =(-1, -1)):
2      """_Fonction_ qui _renvoie_ le _produit_ matriciel _des_ matrices _m1_ et _m2_"""
3      assert(n1[1] == n2[0])
4      if n1 == (-1, -1):
5          n = len(m1)
6          p = n
7          q = n
8      else:
9          n, p = n1[0], n1[1]
10         q = n2[1]
11     res = [[0 for _ in range(q)] for _ in range(n)]
12     for i in range(n):
13         for j in range(q):
14             somme = 0
15             for k in range(p):
16                 somme += m1[i][k] * m2[k][j]
17             res[i][j] = somme
18     return res
19
20 def somme_matricielle(m1: list[list[int]], m2: list[list[int]], n: int, p: int):
21     """_Fonction_ qui _renvoie_ la _somme_ des _matrices_ _m1_ et _m2_"""
22     res = [[0 for _ in range(p)] for _ in range(n)]
23     for i in range(n):
24         for j in range(p):
25             res[i][j] = m1[i][j] + m2[i][j]
26     return res
```

Annexe - fonctions __utiles.py

```
1  def bin_vers_dec(x:str):
2      """_Fonction_ qui_ convertit_ un_ élément_ de_ type_ int_ binaire_ en_ élément_ de_ type_
        int_ décimal"""
3      x_str = str(x)
4      x_liste = [int(c) for c in x_str]
5      x_liste.reverse()
6      x_int = 0
7      k = 0
8      for c in x_liste:
9          x_int += 2**k * c
10         k += 1
11
12     return x_int
13
14 def remplir_zeros(x, n: int):
15     """_Rajoute_ des_ zéros_ jusqu'_à_ ce_ que_ 'x'_ soit_ un_ nombre_ binaire_ codé_ sur_ n_ bits
        _"""
16     x_str = str(x)
17     while len(x_str) < n:
18         x_str = "0" + x_str
19
20     return x_str
```

Annexe - fonctions __utiles.py

```
1  def affiche_matrice(m: list[list]):
2      """Fonction qui affiche une matrice carrée"""
3      for i in range(len(m)):
4          for j in range(len(m)):
5              print(m[i][j], " ", end="")
6          print()
7
8  def decale_liste(liste: list, d: int):
9      """Fonction qui décale les éléments de la liste de 'd' vers la droite
10     et renvoie le résultat"""
11      liste_bis = liste.copy()
12      liste_finale = []
13      d_bis = d
14
15      if d == 0:
16          return liste
17
18      while d_bis > 0:
19          liste_finale = []
20          liste_finale.append(liste_bis.pop())
21          for el in liste_bis:
22              liste_finale.append(el)
23          liste_bis = liste_finale.copy()
24          d_bis -= 1
25
26      return liste_finale
```